

ThreeDViewport

License [GPLv3](#)

A native macOS 3D animation tool built on Metal. Load `.glb` / `.gltf` models, animate them with a keyframe timeline (objects, model groups, camera, and lights), arrange the shot from a Director's POV in Scene mode, apply a video-feedback echo and color grade pass, and export to ProRes `.mov` for compositing.

Requirements

- macOS 14 or later
 - Apple Silicon or Intel Mac with Metal support
 - Swift 5.9 / Xcode 15 or later
-

Building

`swift build` alone does **not** produce a working `.app` bundle. SPM copies Metal shader sources into the resources bundle but never compiles them, and `makeDefaultLibrary(bundle:)` requires a pre-built `default.metallib` — without it the viewport launches white.

Use the included script:

```
bash
# Release build (default) – produces ThreeDViewport.app in the project root
./make_app.sh

# Debug build
./make_app.sh debug

# Launch
open ThreeDViewport.app
```

The script runs the full pipeline in one pass:

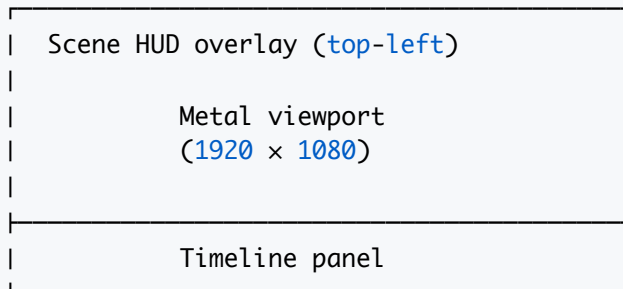
1. `swift build -c release` (or `debug`)
2. Assembles the `.app` bundle (`Contents/MacOS` / `Resources` / `Frameworks`)
3. Copies the executable and the SPM-generated resources bundle
4. Compiles all `.metal` files in `Sources/ThreeDViewport/Renderer/` into `default.metallib` via

6. Writes `Info.plist`

7. Ad-hoc code-signs the bundle (`codesign --sign -`) so macOS will run it

New `.metal` files are picked up automatically. For distribution with a Developer ID, replace the `-` in the script's `codesign` invocation with your certificate name.

Window layout



- **Metal viewport** — the 3D render area. Receives all mouse and keyboard input when focused.
- **Scene HUD** — semi-transparent overlay showing the active control mode, the object list with visibility toggles, and the selected item.
- **Timeline panel** — transport controls, playhead scrubber, duration popover, “Add Key” button, and Export button along the bottom.

Floating inspector panels open independently and remember their positions per project. They do not steal keyboard focus from the viewport (except the Timeline Editor, which becomes key on click so its shortcuts work). When the main window is minimized, all open panels hide together and reappear when the window is restored.

Panel	Menu	What it does
Camera	Window → Camera... (⌘K)	Pick a follow target from a dropdown and stamp follow keyframes — see “Camera follow keyframes” below
Lights & Background	Window → Lights & Background... (⌘L)	Edit up to four lights, plus solid / gradient background
Feedback	Window → Feedback... (⌘F)	Video delay-line echo / trails pass
Color Grade	Window → Color Grade... (⌘⇧G)	Brightness / contrast / gamma post-process
Timeline Editor	Window → Timeline Editor (⌘J)	Per-track keyframe editor with retiming, copy/paste, easing modes, and in-place edit

What you can do

Load models

Open `.glb` (single binary) or `.gltf` + `.bin` (JSON + sidecar) via **File** → **Open Model...** (⌘O). Multiple models can be loaded; each is auto-normalised to a 1-unit bounding sphere on first load. The camera fits to the first model; additional models append without resetting the camera. Hierarchical rigs (e.g. a character with named bones) preserve their parent-child structure for FK animation.

Animate with keyframes

Four kinds of animation, each with its own timeline track:

- **Object** — TRS deltas on a single SceneObject. Stamped in **Object** mode (`O`).
- **Model group** — TRS applied as a single layer on top of every part in a multi-part model (e.g. translate / rotate a whole robot as one). Stamped in **Model** mode (`M`).
- **Camera** — absolute yaw / pitch / distance / target / FOV. Stamped via the Timeline panel's "Add Key" or the Camera panel.
- **Light** — per-light intensity, colour, direction, position, range, and beam thickness. Stamped in **Light** mode (`L`).

Object and group tracks support several easing modes (Linear plus Catmull-Rom splines at three tensions); camera and light tracks always interpolate linearly. The "I" key (or Insert) stamps a keyframe for whichever mode is active.

Camera follow keyframes

A follow keyframe ties the camera to a named object (e.g. `head`) so the camera holds its **relative** position and orientation as the object moves, rotates, and rolls. Workflow:

1. Open the **Camera panel** (⌘K) and pick the target object from the dropdown.
2. Frame the shot from the recording camera.
3. Click **Add Follow Camera Keyframe**. Stamp as many as you like — the picker stays sticky on the chosen target.

Between two follow keyframes targeting the same object, both the camera's target offset and its forward direction are interpolated in the object's **local** frame, then rotated by the object's current basis each frame. The camera holds its framing under any object rotation (yaw, pitch, roll, combined) without gimbal-lock issues.

Scene mode (Director's POV)

Press **S** to toggle Scene mode. The viewport switches from "look through the scene camera" to "look at the scene camera from above and behind" — a free Director camera. The scene camera is drawn as

follow paths, debugging framing, and recording reference shots. **R** re-auto-fits the Director to the scene; **+** / **-** dolly the Director in / out; Space + drag orbits it.

Lights & background

Up to four simultaneous lights, each independently configurable:

Type	Notes
Ambient	Flat fill, no direction or position
Directional	World-space direction; classic sun light
Point	Position only; falls off with distance
Spot	Position + direction + inner / outer cone angles + range
Laser	Narrow beam, depth-occluded by geometry; optional thick billboard with Gaussian glow; can be excluded from the feedback pass

Background is either a solid colour or a vertical gradient (top / bottom colours).

Feedback delay-line

A ring-buffered echo effect that blends each rendered frame with a delayed copy. Tunable blend mode (Normal / Multiply / Screen / Additive / Overlay / Soft Light / Difference / etc.), feedback-on-top swap, decay weight, capture interval, and ring length. Status badge shows whether the buffer is priming or active. Runs only during playback and during export.

Color grade

A final-pass shader applied to every rendered frame and to the export. Brightness → Contrast → Gamma, with a Reset button when anything is off identity. The pass is bypassed entirely when all three controls are at their identity values, so there's no cost when unused.

Video export

File → Export ProRes Video... (**E**) renders the full timeline offline to a 1920 × 1080 **.mov** :

Codec	Notes
ProRes 4444 — Alpha = Luma	12-bit RGB + alpha; alpha is the Rec.709 luma of each pixel. Composite directly in DaVinci Resolve or LZX Videomancer without a separate key pass.
ProRes 422 HQ — solid black	10-bit 4:2:2, no alpha. Standard for CG renders going to a colour-grading pipeline.

The exporter runs on its own GPU pipeline with a fresh feedback processor for repeatable output. The viewport pauses during export and resumes when done; transport controls are disabled while a render is in flight.

Project files

Projects are saved as `.3dvp` files (human-readable JSON, current version **14**). The schema is additive — older files load cleanly because unknown / missing keys are silently ignored and fields with defaults backfill themselves.

Saved state covers everything needed to reproduce a session:

- Timeline duration and loop-playback setting
- Camera pose + keyframe track (including follow-target metadata and local-frame forward vectors)
- All loaded model paths, per-object base transforms, per-object keyframe tracks (with easing mode), and per-group keyframe tracks
- Light configurations + per-light keyframe tracks
- Background, color-grade, feedback, rendering-mode (color / greyscale), wireframe, and axes-gizmo settings
- Window and panel positions for the main window, Timeline Editor, and every inspector panel — restored on load

Model files are referenced by absolute path; they need to remain at their original location for the project to reload cleanly. If a referenced model is missing on load, the project prompts you to locate it.

Supported model formats

- glTF 2.0 Binary (`.glb`) — single self-contained file with embedded geometry and textures.
- glTF 2.0 JSON (`.gltf` + `.bin`) — JSON descriptor with external binary buffers. The `.bin` file(s) must remain alongside the `.gltf`.

Both support PBR materials (base colour, normal, metallic / roughness, emissive), per-vertex colour fallback, embedded or external meshes, and node hierarchies. If a model fails to load (e.g. Draco compression, an unsupported extension), an alert describes the reason.

Keyboard and mouse

See `KEYBOARD_REFERENCE.md` for the complete list of viewport, timeline-editor, and menu shortcuts.

